Runbook — Basemap Offline (Estado de São Paulo)

Guia oficial para carga e publicação do landbase **SP completo** (escala estadual). Pipeline versionado e repetível — referência principal para o produto.

> **Deploy completo da stack:** ver `docs/mds/DEPLOY_POC.md` **Estado operacional actual:** ver `docs/mds/CONTEXT_STATE.md`

---

1. Objectivo

- Disponibilizar fundo cartográfico offline e livre para operação de utilities em todo o estado de SP
- Garantir reprodutibilidade (script único, sem passos manuais)
- Manter compatibilidade com GeoServer (WMS/WFS) e evolução futura para tiles vectoriais

---

2. Pré-requisitos

Stack activa (`db`, `geoserver`) via `infra/docker-compose.yml`:

```
docker compose -f infra/docker-compose.yml up -d db geoserver
```

Ferramentas no host: `psql`, `shp2pgsql`, `osm2pgsql`, `curl`, `python3`

---

3. Entradas de dados

Ficheiro	Origem	Observação
`data/raw/ibge/BR_UF_2024.zip`	IBGE	Shapefile UF Brasil
`data/raw/ibge/BR_Municipios_2024.zip`	IBGE	Shapefile municípios Brasil
`data/raw/osm/sao-paulo-latest.osm.pbf`	Geofabrik	**Preferido** — extrato estadual
`data/raw/osm/sudeste-latest.osm.pbf`	Geofabrik	**Fallback automático** — usado se o PBF estadual for inválido

O script descarrega automaticamente os ficheiros IBGE se ausentes. O download do OSM deve ser feito manualmente (tamanho elevado):

```
# Extrato SP (~100 MB)
curl -fL -o data/raw/osm/sao-paulo-latest.osm.pbf \
  https://download.geofabrik.de/south-america/brazil/sudeste/sao-paulo-latest.osm.pbf
```

---

4. Execução

Na raiz do repositório:

```
./infra/scripts/setup_landbase_sp_estado.sh
```

O script é idempotente — pode ser re-executado sem duplicar dados.

---

5. Entregáveis

PostGIS (schema `landbase`)

Tabela	Conteúdo	SRID
`landbase.uf_sp`	Polígono UF SP (IBGE)	EPSG:4674
`landbase.municipios_sp`	Municípios com geometria intersectando UF SP	EPSG:4674
`landbase.vias_sp`	Linhas OSM com tag `highway`, recortadas ao estado	EPSG:4674

Tabelas intermediárias `public.osm_sp_*` (prefixo `osm_sp`) geradas pelo `osm2pgsql`.

GeoServer (workspace `base_sp`)

Camada	Estilo SLD
`base_sp:uf_sp`	`base_sp_uf_outline`
`base_sp:municipios_sp`	`base_sp_municipios_outline`
`base_sp:vias_sp`	`base_sp_vias_zoom_scales`

Estilos SLD versionados em `infra/geoserver/styles/`.

5.1 Recorte prioritário (municípios-alvo, bairros, vias)

O landbase estadual (secção 5) cobre **todo o estado**. Para a POC TabGis usa-se um **recorte metropolitano** (24 municípios) com camadas adicionais e estilos dedicados.

Passo	Comando / referência
Gerar tabelas e publicar camadas	`./infra/scripts/setup_municipios_alvo_sp.sh`
Detalhe da lista de municípios e comportamento no mapa	`docs/mds/CONTEXT_STATE.md` → *Recorte prioritário* e *Frontend*

PostGIS (além das tabelas da secção 5)

Tabela	Conteúdo
`landbase.municipios_alvo_sp`	Polígonos dos 24 municípios-alvo
`landbase.bairros_alvo_sp`	Bairros (OSM) intersectando os municípios-alvo
`landbase.vias_alvo_sp`	Vias públicas no mesmo recorte

GeoServer — camadas e SLD em repositório

Camada	SLD versionado (repo)	Notas
`base_sp:municipios_alvo_sp`	`infra/geoserver/styles/base_sp_municipios_alvo_highlight.sld`	Rótulos `nm_mun` visíveis nas escalas usadas na POC

Camada	SLD versionado (repo)	Notas
`base_sp:bairros_alvo_sp`	`infra/geoserver/styles/base_sp_bairros_alvo.sld`	Preenchimento/contorno discretos; rótulos `municipio_nome` + `bairro_nome`
`base_sp:vias_alvo_sp`	(estilo atribuído pelo script; alinhado a vias por zoom onde aplicável)	—

Manutenção: alterações a estes `.sld` no Git **não** entram automaticamente no GeoServer após `git pull`. Republicar o XML na consola (**Styles** / camada) ou repetir o procedimento de publicação do teu ambiente. Sincronismo com toggles e zoom do cliente: `docs/mds/CONTEXT_STATE.md` (*Camadas alvo — zoom e toggles*, *Estilos GeoServer — recorte alvo*).

5.2 Sanidade `bairros_alvo_sp` (pós-regeneração)

Após `./infra/scripts/setup_municipios_alvo_sp.sh`, as geometrias OSM podem fazer com que o nome de **outro município** (ex.: Osasco) apareça como falso "bairro" num recorte (ex.: Cotia), poluindo o dropdown no cliente. A verificação versionada alinha o critério de correção em `infra/sql/19_fix_bairros_alvo_mapping.sql` e deve devolver **0** linhas.

```
# Na raiz do repositório (ajusta PGPASSWORD / host se necessário)
PGPASSWORD="${DB_PASS:-password123}" psql -h "${DB_HOST:-localhost}" -p "${DB_PORT:-5433}" \
  -U "${DB_USER:-tabgis_admin}" -d "${DB_NAME:-tabgis_db}" -v ON_ERROR_STOP=1 \
  -f infra/sql/20_sanity_bairros_alvo_sp.sql
```

- Se o comando **falhar** (código de saída 0), listar as violações no output, aplicar `infra/sql/19_fix_bairros_alvo_mapping.s` se for aceitável e voltar a correr a sanidade, ou ajustar a carga OSM. O frontend continua a filtrar nomes de município na lista, mas a fonte em PostGIS deve estar limpa para WFS/rotulagem.

---

6. Validação

```
# WMS GetCapabilities
curl -s "http://localhost:8080/geoserver/wms?service=WMS&request=GetCapabilities" \
  | grep -o 'base_sp:[^<]*' | head

# Contagem PostGIS
PGPASSWORD=password123 psql -h localhost -p 5433 -U tabgis_admin -d tabgis_db -c \
  "SELECT 'uf_sp' AS tabela, COUNT(*) FROM landbase.uf_sp
   UNION ALL SELECT 'municipios_sp', COUNT(*) FROM landbase.municipios_sp
   UNION ALL SELECT 'vias_sp', COUNT(*) FROM landbase.vias_sp;"
```

---

7. Escalabilidade

- Para outro estado: alterar filtro `sigla_uf` em `landbase.uf_sp`, ajustar `SP_BBOX` (ou usar PBF do estado), re-executar
- Variáveis configuráveis: `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASS`, `GEOSERVER_URL`, `GS_USER`, `GS_PASS`, `OSM_SP`, `OSM_SUDESTE`, `SP_BBOX`